

Vizuálny transformer

28.1.2026, 15:00-17:00

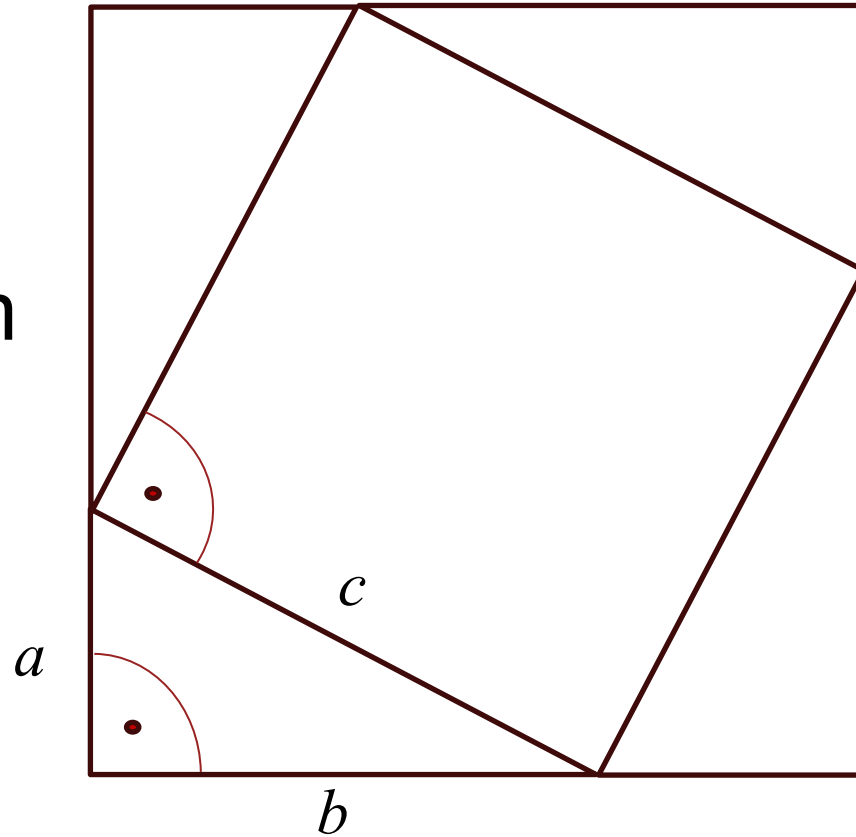
Andrej Lúčny

Kosínusová podobnosť. Vnorenie a vynorenie.
Pozornostný mechanizmus. Vizuálny transformer (ViT).

<https://github.com/andy1ucny/OAI>

Pythagorova veta

v pravouhlom
trojuholníku:



$$(a + b)^2 = c^2 + 4 \frac{ab}{2}$$

$$\implies$$

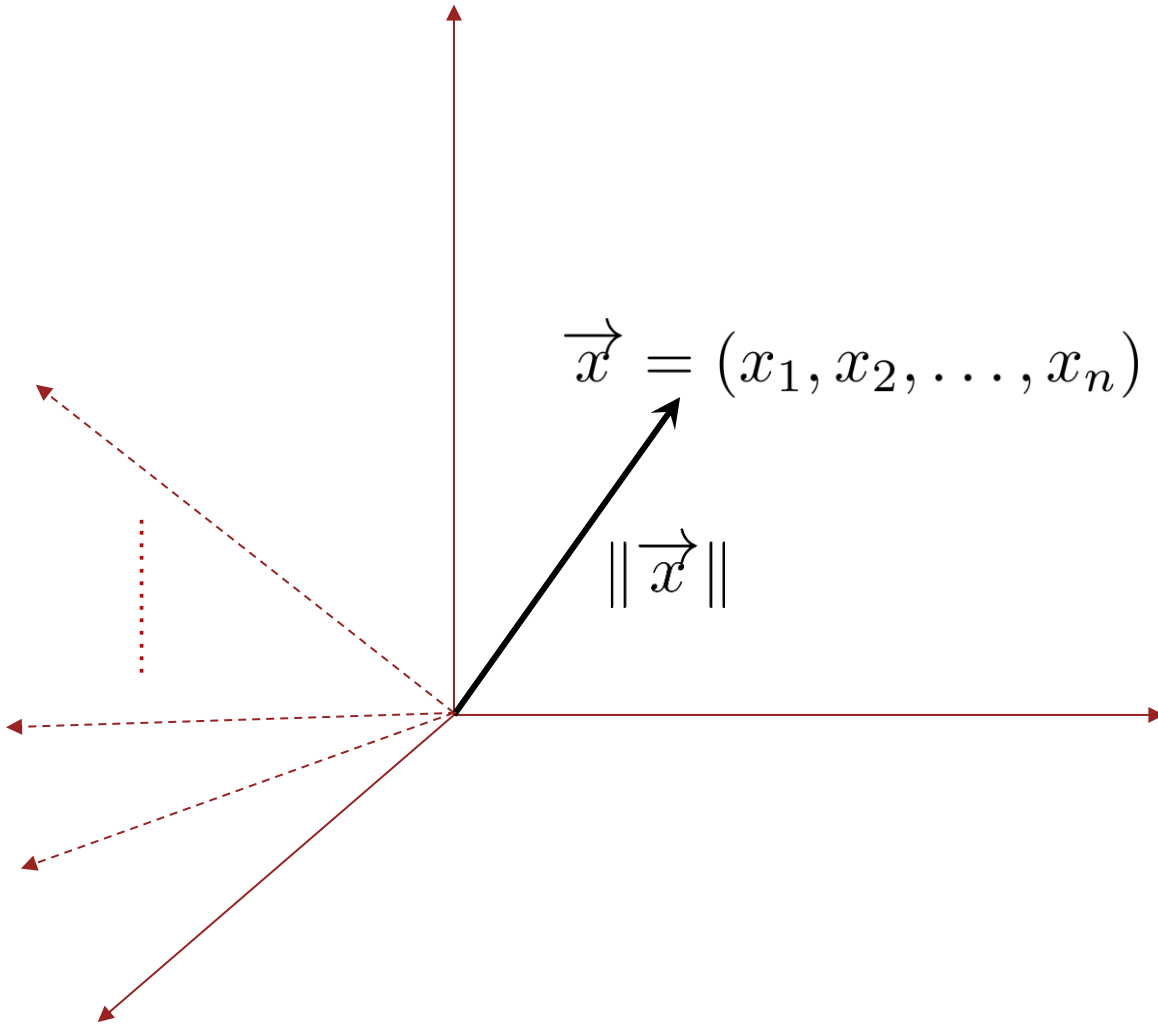
$$a^2 + 2ab + b^2 = c^2 + 2ab$$

$$\implies$$

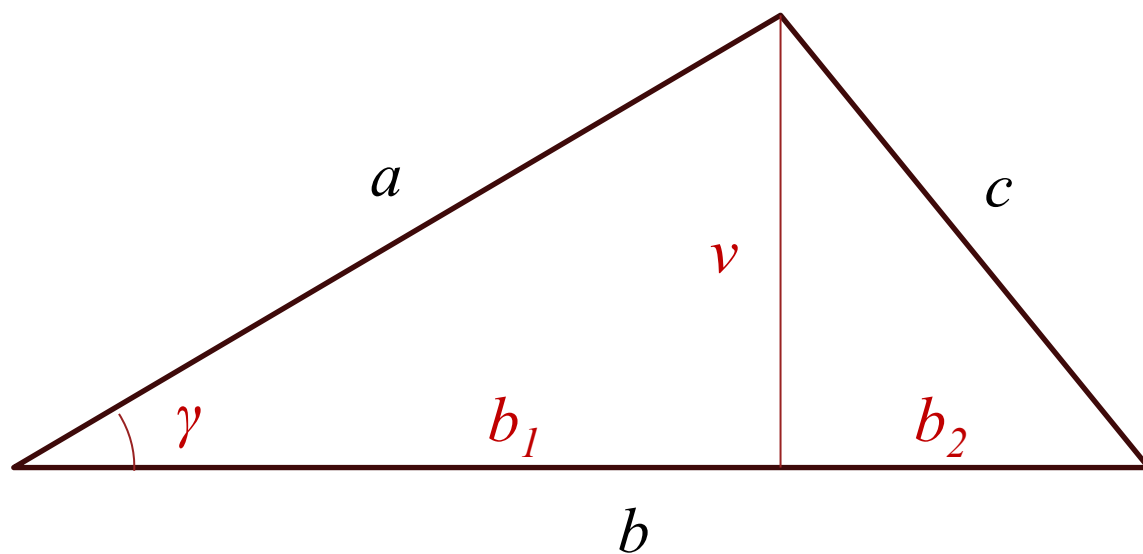
$$a^2 + b^2 = c^2$$

dĺžka vektora
v n-rozmernom priestore:

$$\|\vec{x}\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$



Kosínusová veta



$$\begin{aligned}v^2 &= a^2 - b_1^2 \\v^2 &= c^2 - b_2^2 = c^2 - (b - b_1)^2 = \\&= c^2 - b^2 + 2bb_1 - b_1^2\end{aligned}$$

$$\begin{aligned}a^2 - b_1^2 &= c^2 - b^2 + 2bb_1 - b_1^2 \\a^2 &= c^2 - b^2 + 2bb_1 \\c^2 &= a^2 + b^2 - 2bb_1 \\c^2 &= a^2 + b^2 - 2ab \frac{b_1}{a}\end{aligned}$$

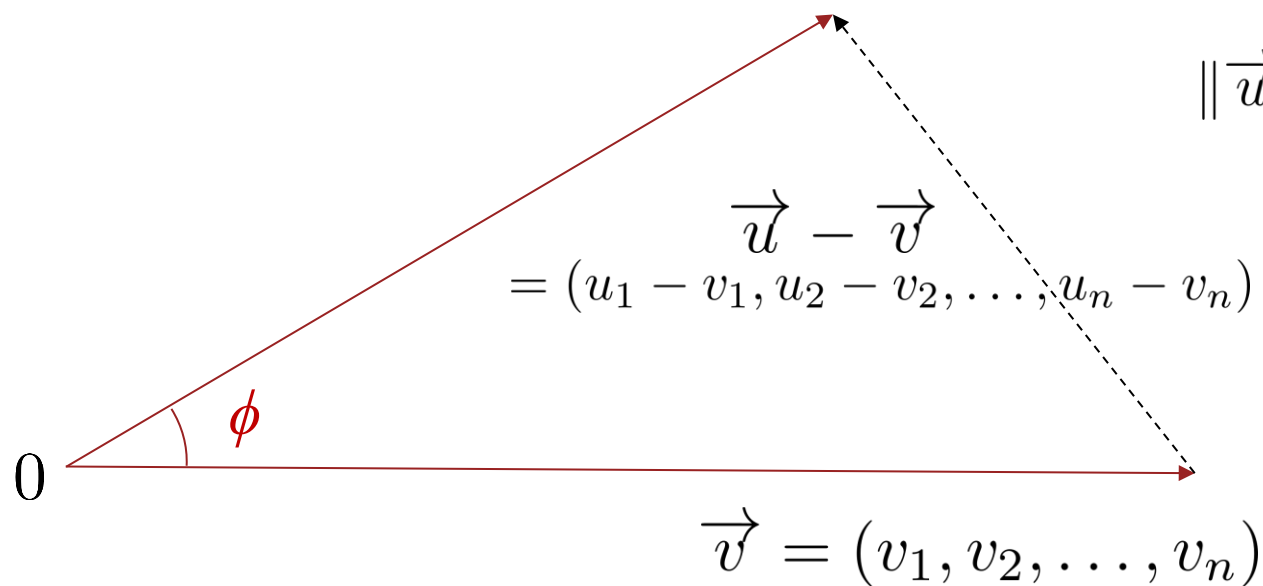
$\cos \gamma$

$$c^2 = a^2 + b^2 - 2ab \cos \gamma$$

Kosínusová veta pre vektory

$$\vec{u} = (u_1, u_2, \dots, u_n)$$

$$\|\vec{u} - \vec{v}\|^2 = \|\vec{u}\|^2 + \|\vec{v}\|^2 - 2\|\vec{u}\|\|\vec{v}\|\cos\phi$$



$$\cos\phi = \frac{\|\vec{u}\|^2 + \|\vec{v}\|^2 - \|\vec{u} - \vec{v}\|^2}{2\|\vec{u}\|\|\vec{v}\|}$$

$$\cos \phi =$$

$$= \frac{\|\vec{u}\|^2 + \|\vec{v}\|^2 - \|\vec{u} - \vec{v}\|^2}{2\|\vec{u}\|\|\vec{v}\|} =$$

$$= \frac{u_1^2 + u_2^2 + \dots + u_n^2 + v_1^2 + v_2^2 + \dots + v_n^2 - (u_1 - v_1)^2 - (u_2 - v_2)^2 - \dots - (u_n - v_n)^2}{2\|\vec{u}\|\|\vec{v}\|} =$$

$$= \frac{u_1^2 + u_2^2 + \dots + u_n^2 + v_1^2 + v_2^2 + \dots + v_n^2 - u_1^2 + 2u_1v_1 - v_1^2 - u_2^2 + 2u_2v_2 - v_2^2 - \dots - u_n^2 + 2u_nv_n - v_n^2}{2\|\vec{u}\|\|\vec{v}\|} =$$

$$= \frac{2u_1v_1 + 2u_2v_2 + \dots + 2u_nv_n}{2\|\vec{u}\|\|\vec{v}\|} =$$

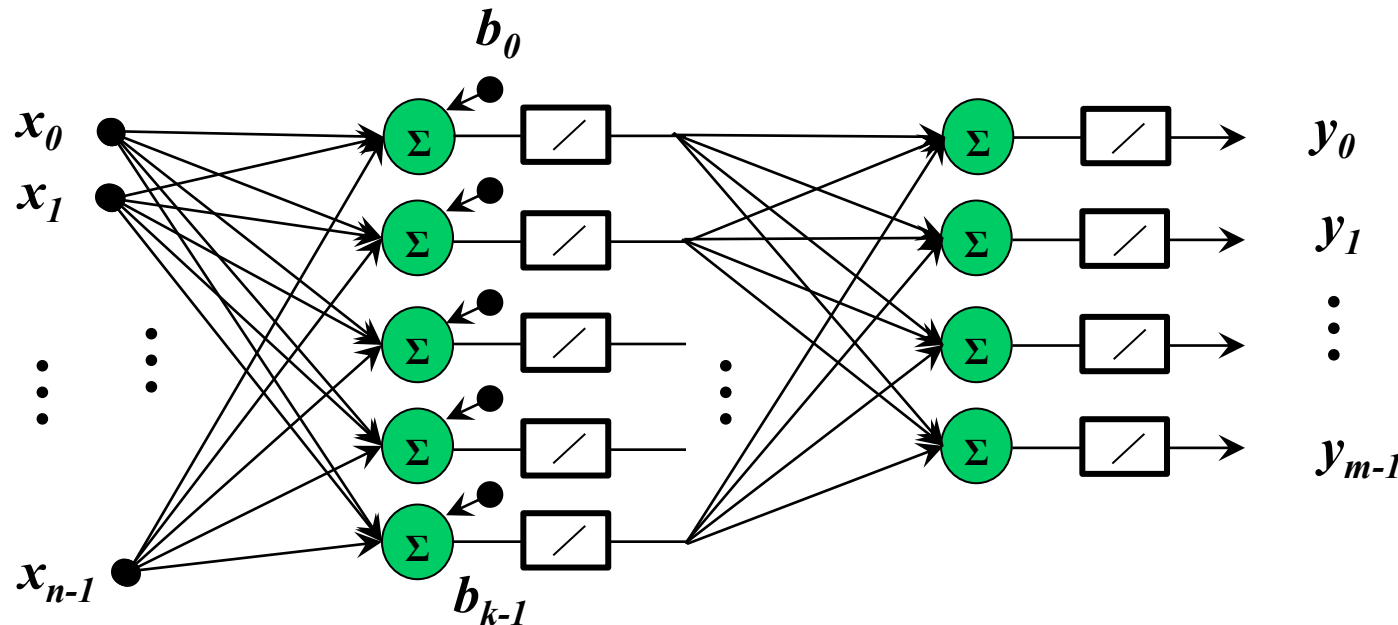
$$= \frac{u_1v_1 + u_2v_2 + \dots + u_nv_n}{\|\vec{u}\|\|\vec{v}\|} = \vec{u} \cdot \vec{v}$$

skalárny súčin

kosínusová veta pre vektory:

$$\cos \phi = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|\|\vec{v}\|}$$

Perceptrón



Vezmime si najjednoduchší perceptrón:

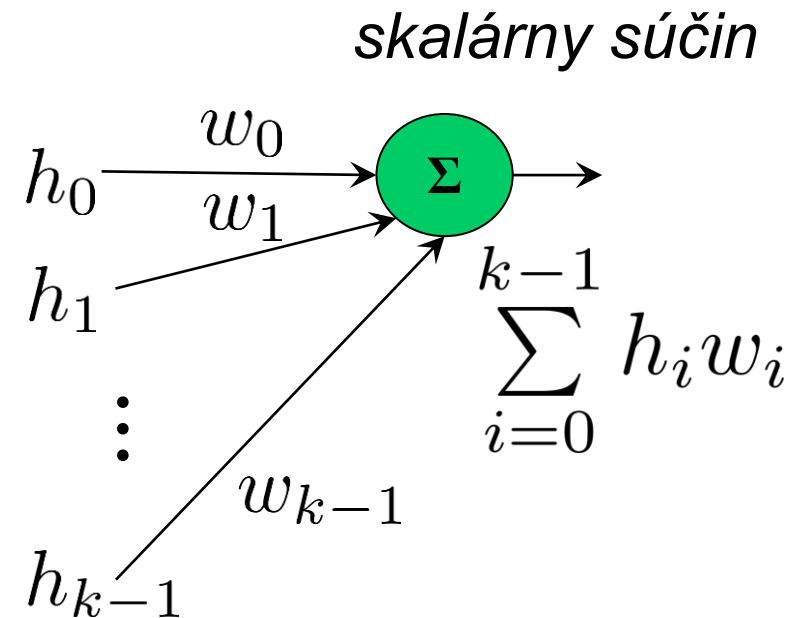
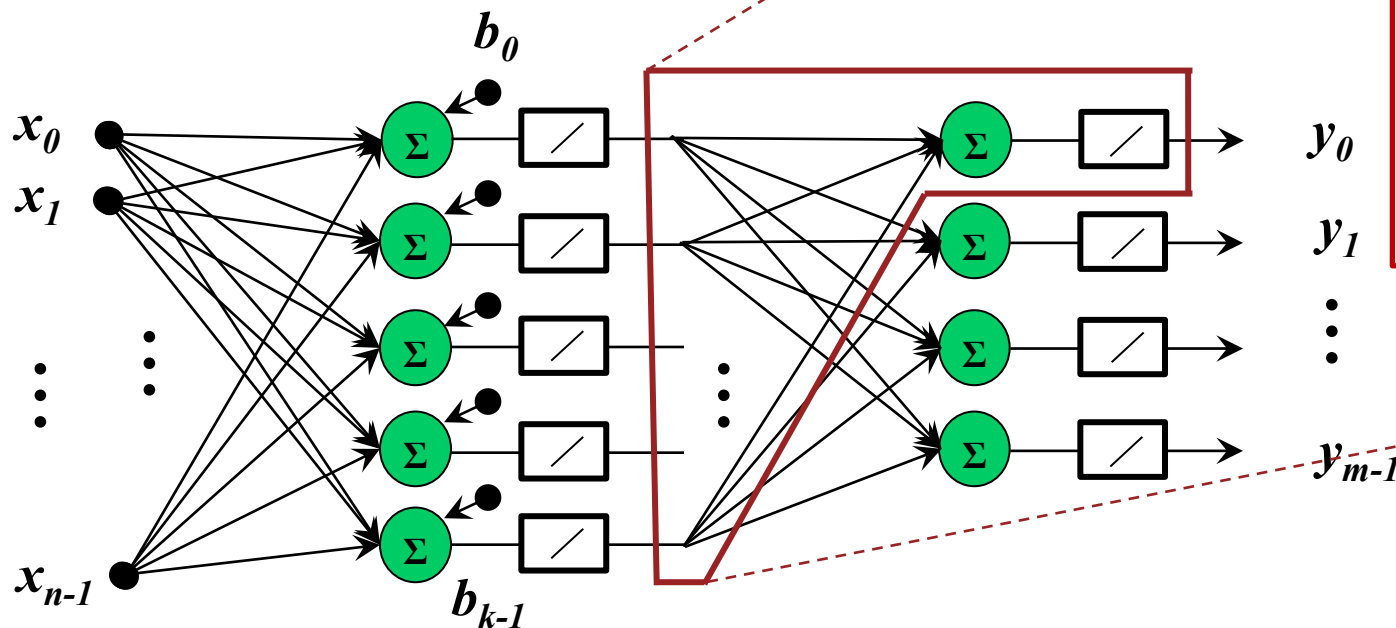
- Lineárna aktivácia
- Biás vo výstupnej vrstve nulový

- takýto model nie je univerzálny aproximátor, ale môžeme sa ho pokúsiť natrénovať

- Úloha: napríklad klasifikácia cifier datasetu MNIST 
1 3 4 9 1 5 0 7 4 0

- vstup: $n=784$ intenzít pixelov 28×28
- výstup: $m=10$ logitov
- $k=1568$ skrytých neurónov
- chybová funkcia: BCE
- maximálny logit zodpovedá kategórii

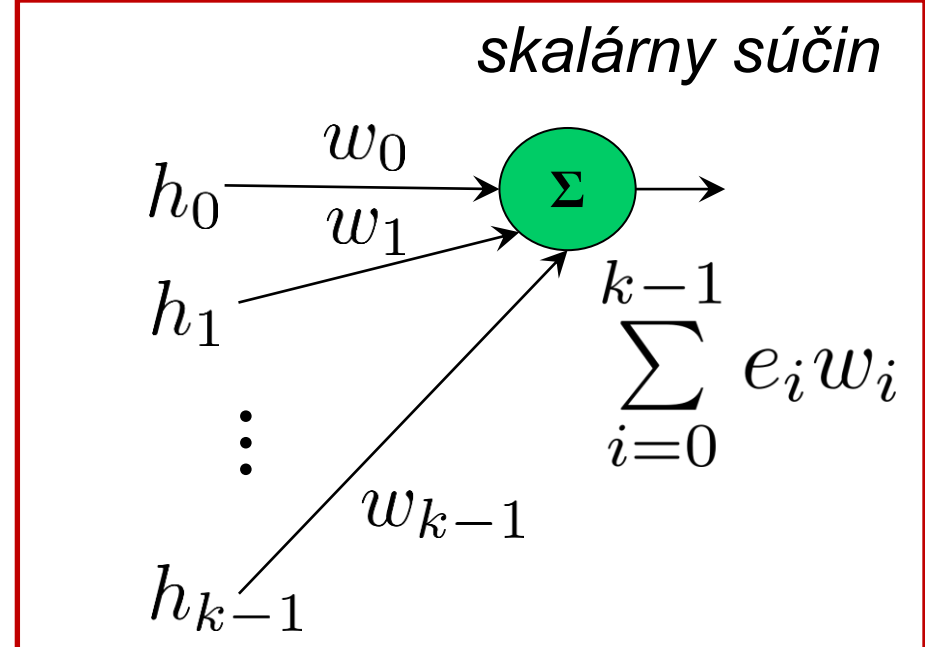
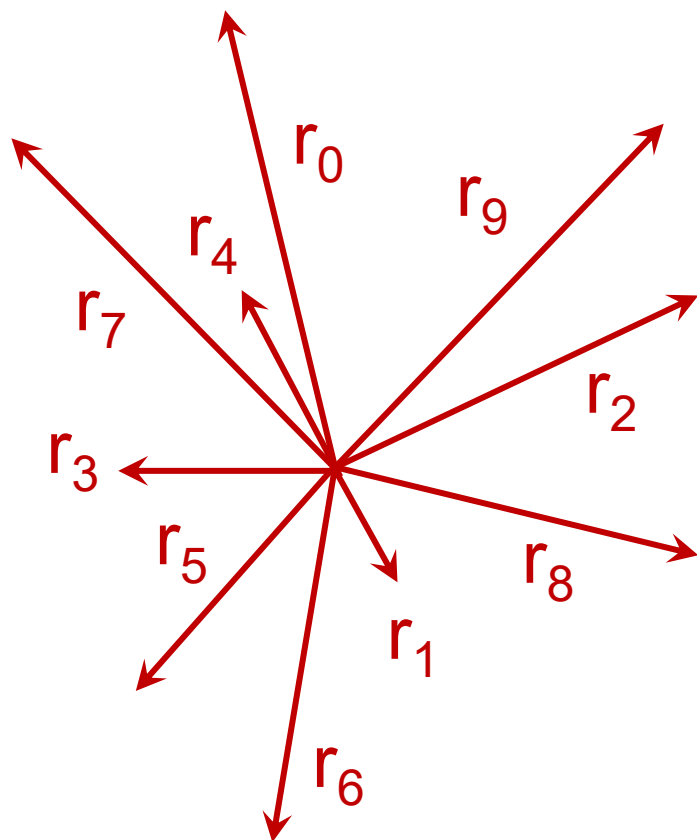
Povaha logitov



Pozrime sa teraz bližšie na výsledok

- Keďže vo výstupnej vrstve nemáme biásy, výstupné logity sú skalárnym súčinom výstupov skrytých neurónov s váhami vo výstupnej vrstve

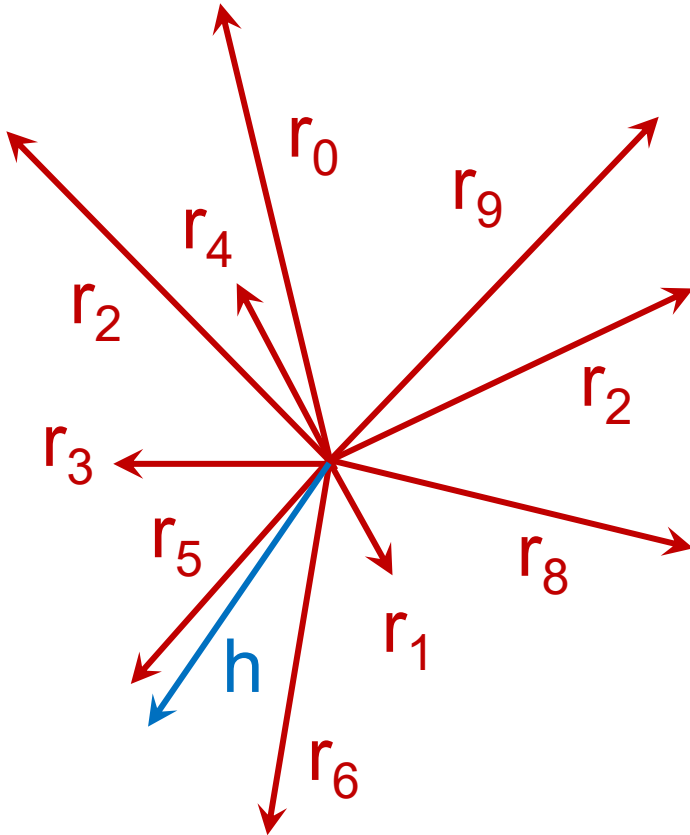
Reprezentanti



- Váhy výstupných neurónov preto možno považovať za m vektorov reprezentujúcich jednotlivé kategórie v n rozmernom priestore

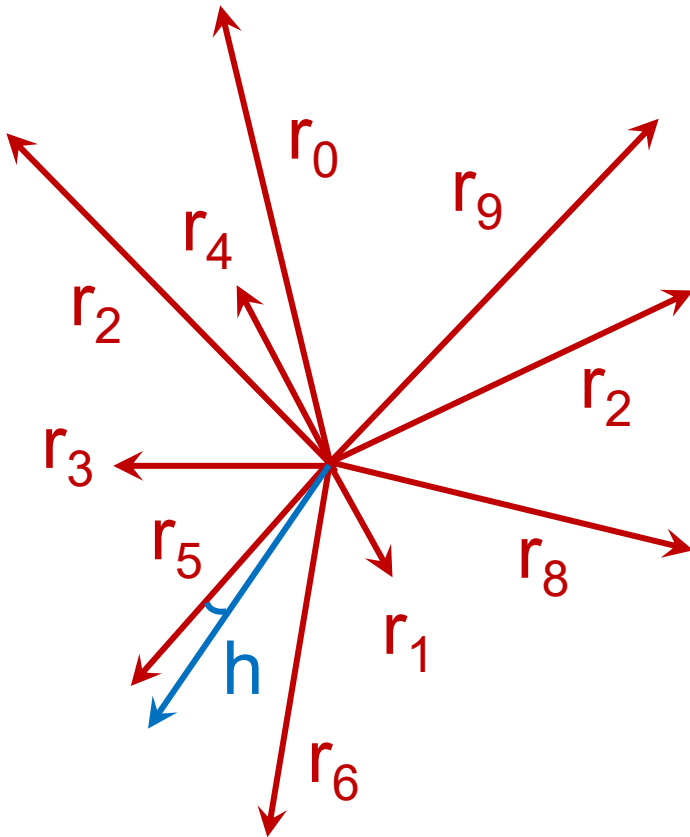
Vynorenie (Wipeout)

wipe out = vytriet', vyhubit'



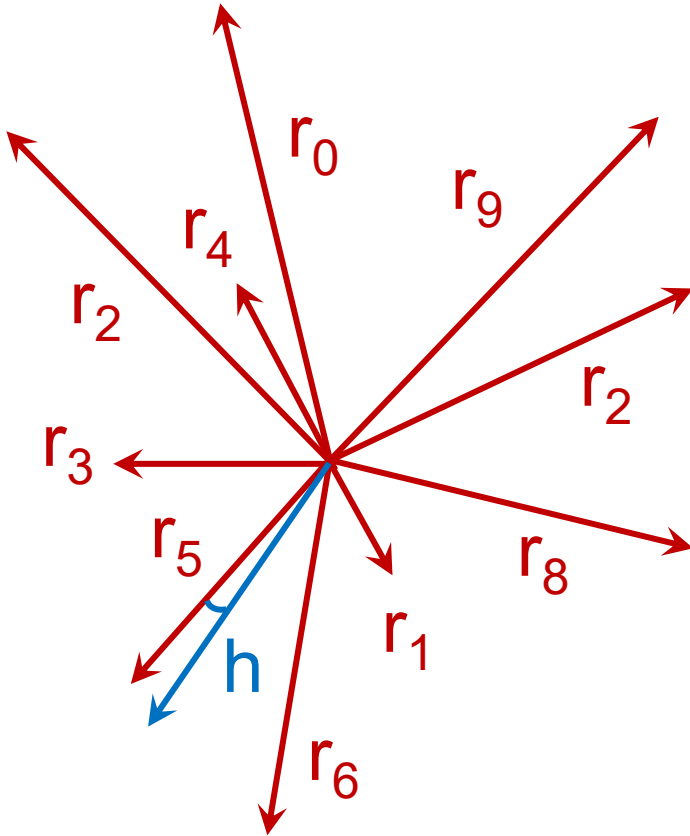
- Keď výstupná vrstva dostane skrytý stav h ,
- spočíta jeho skalárne súčiny s reprezentantmi
- a index reprezentanta, s ktorým je skalárny súčin maximálny, vyhlási za určenú kategóriu

Vynorenie (Wipeout)



- Podľa kosínusovej vety pre vektory je skalárny súčin rovný kosínusu uhla zvieraného vektormi vynásobného dĺžkami vektorov
- Takže ak sú reprezentanti a všetky skryté stavy približne rovnako veľké,
- sieť za určenú kategóriu vyhlási tú, ktorej reprezentant zviaza so skrytým stavom najmenší uhol (kosínusová podobnosť)

Vynorenie (Wipeout)



- Skrytý stav h môžeme nazvať dotazom q (*query*)
- Reprezentantov r_i môžeme nazvať kľúčmi k_i (*keys*)
- Keď hľadáme kategóriu:

$$\arg \max_i q k_i$$

- Keď chceme pravdepodobnosti príslušnosti ku kategóriám:

$$\text{softmax}(q K^T)$$

Maticové zápisy

$$q = (q_0, q_1, \dots, q_{d-1})$$

$$k_i = (k_{i,0}, k_{i,1}, \dots, k_{i,d-1})$$

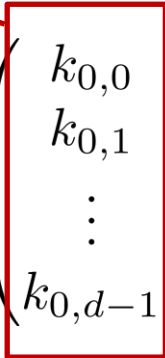
$$K = \begin{pmatrix} k_{0,0} & k_{0,1} & \dots & k_{0,d-1} \\ k_{1,0} & k_{1,1} & \dots & k_{1,d-1} \\ \vdots & \vdots & \ddots & \vdots \\ k_{l-1,0} & k_{l-1,1} & \dots & k_{l-1,d-1} \end{pmatrix}$$

k_0



$$K^T = \begin{pmatrix} k_{0,0} & k_{1,0} & \dots & k_{l-1,0} \\ k_{0,1} & k_{1,1} & \dots & k_{l-1,1} \\ \vdots & \vdots & \ddots & \vdots \\ k_{0,d-1} & k_{1,d-1} & \dots & k_{l-1,d-1} \end{pmatrix}$$

k_0



$$qK^T = (q_0, q_1, \dots, q_{d-1}) \begin{pmatrix} k_{0,0} & k_{1,0} & \dots & k_{d-1,0} \\ k_{0,1} & k_{1,1} & \dots & k_{d-1,1} \\ \vdots & \vdots & \ddots & \vdots \\ k_{0,l-1} & k_{1,l-1} & \dots & k_{d-1,l-1} \end{pmatrix} = (qk_0, qk_1, \dots, qk_{l-1})$$



$$qk_i = \sum_{j=0}^{d-1} q_j k_{i,j}$$

afínna transformácia

$(x_0, x_1, \dots, x_{n-1})$

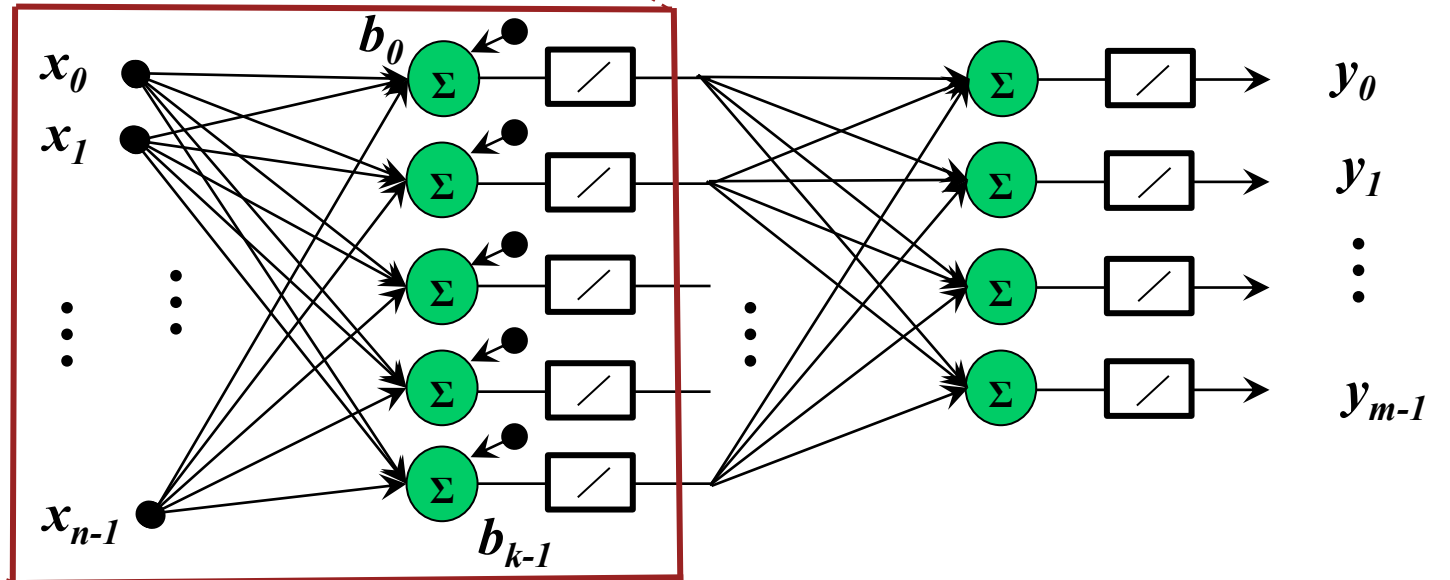


$(h_0, h_1, \dots, h_{k-1})$

$$h_i = \sum_{j=0}^{n-1} x_j w_{i,j} + b_i$$

Vnoreníe (Embedding)

(pre obrazové dáta)



- Skrytý stav môžeme vnímať ako vnútornú reprezentáciu vstupu
- Dimenzia skrytého stavu zodpovedá počtu príznačkov, ktorými sa dva vstupy môžu vzájomne líšiť

Normalizácia v rámci vrstvy (Layer Normalization)

- Keď medzi vnorenie a vynorenie vložíme stavebný prvok, ktorý zabezpečuje aby boli rozsahy a distribúcia jednotlivých príznakov zhruba rovnaké,
- zabezpečí nám to zároveň, že skryté stavy budú zhruba rovnako veľké
- a podobnosť medzi skrytými stavmi bude kosínusová

trénovateľné parametre

$(w_0, w_1, \dots, w_{k-1})$ $(b_0, b_1, \dots, b_{k-1})$

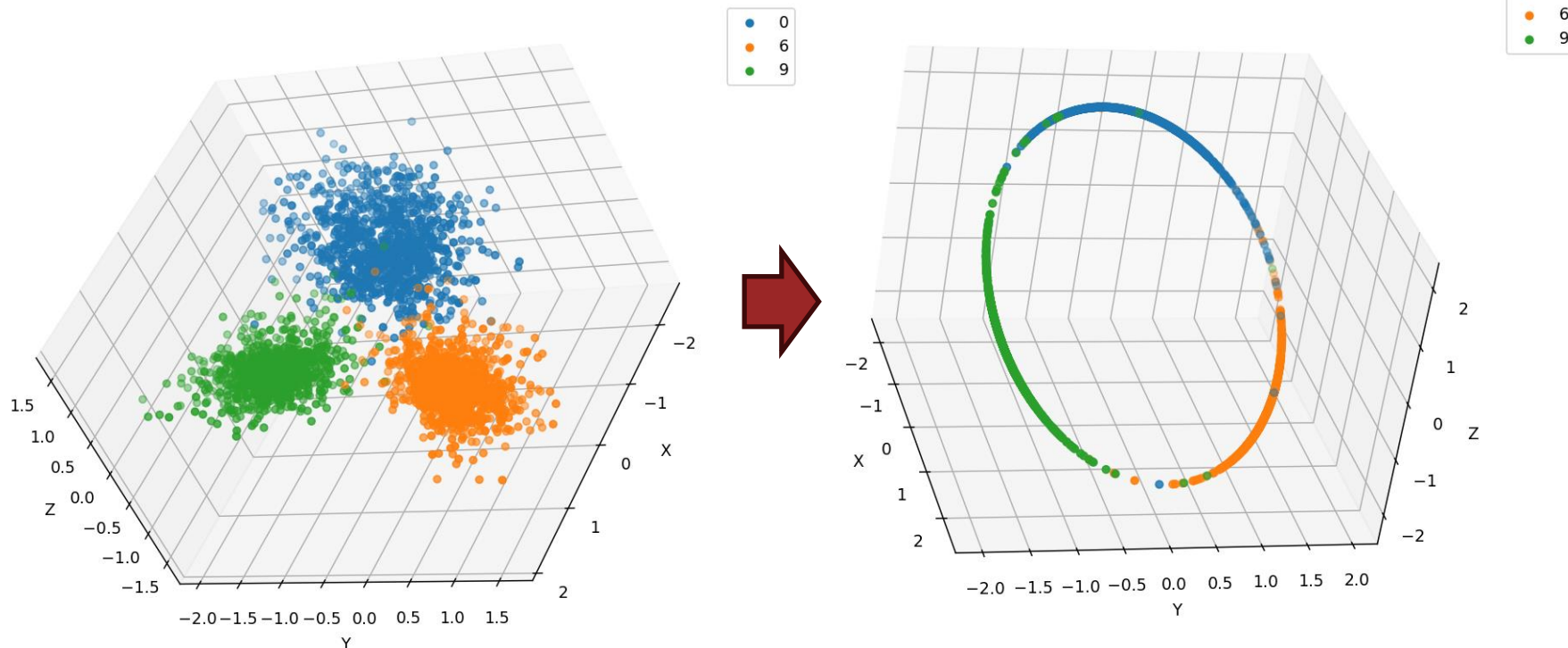
$$LN(h) = w \frac{h - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}} + b$$

$$\mu_L = \frac{1}{k} \sum_{i=0}^{k-1} h_i$$

$$\sigma_L^2 = \frac{1}{k} \sum_{i=0}^{k-1} (h_i - \mu_L)^2$$

Normalizácia v rámci vrstvy (Layer Normalization)

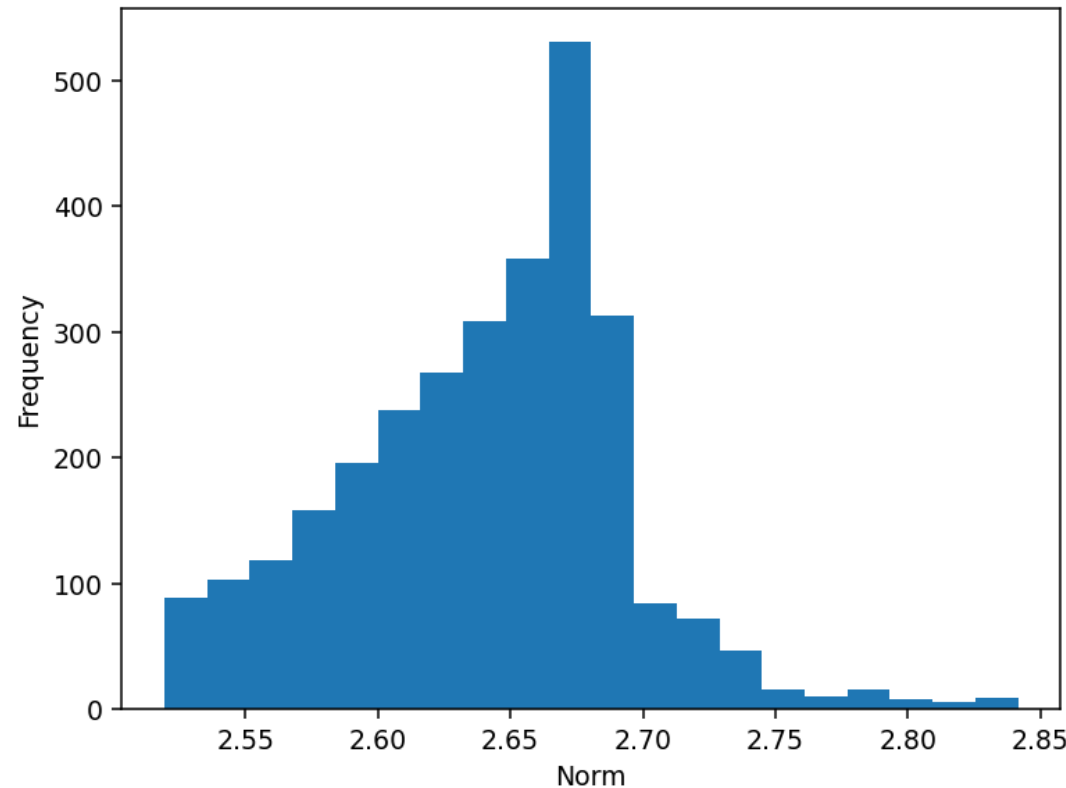
- zobrazuje vzorové vstupy na hyperguľu
- pre $d=3$ na kružnicu, pre $d=4$ na guľu, ...



$d = 3$

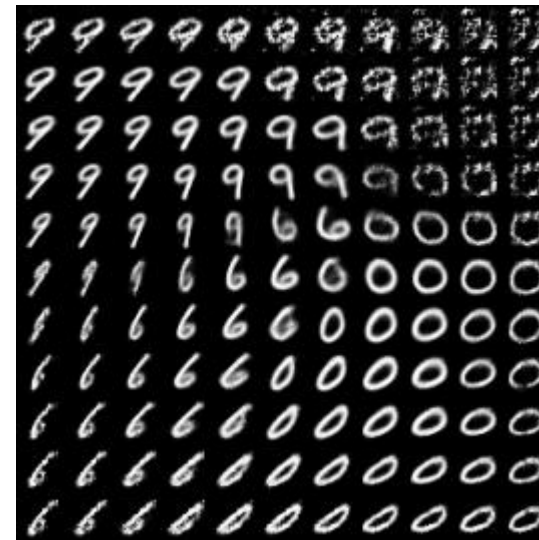
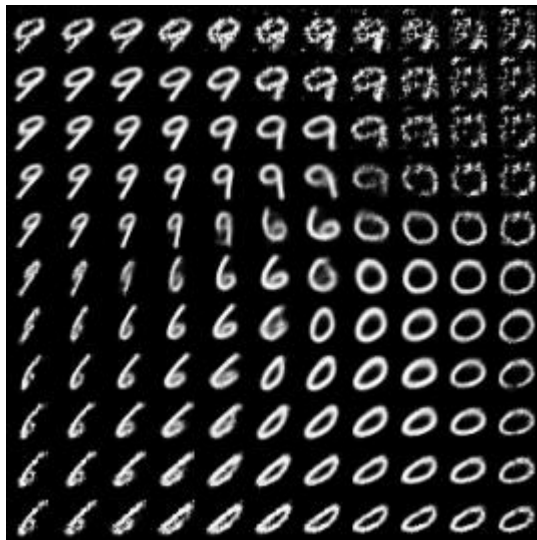
Normalizácia v rámci vrstvy (Layer Normalization)

- príznakové vektory nie sú rovnako dlhé, ale podobne dlhé sú



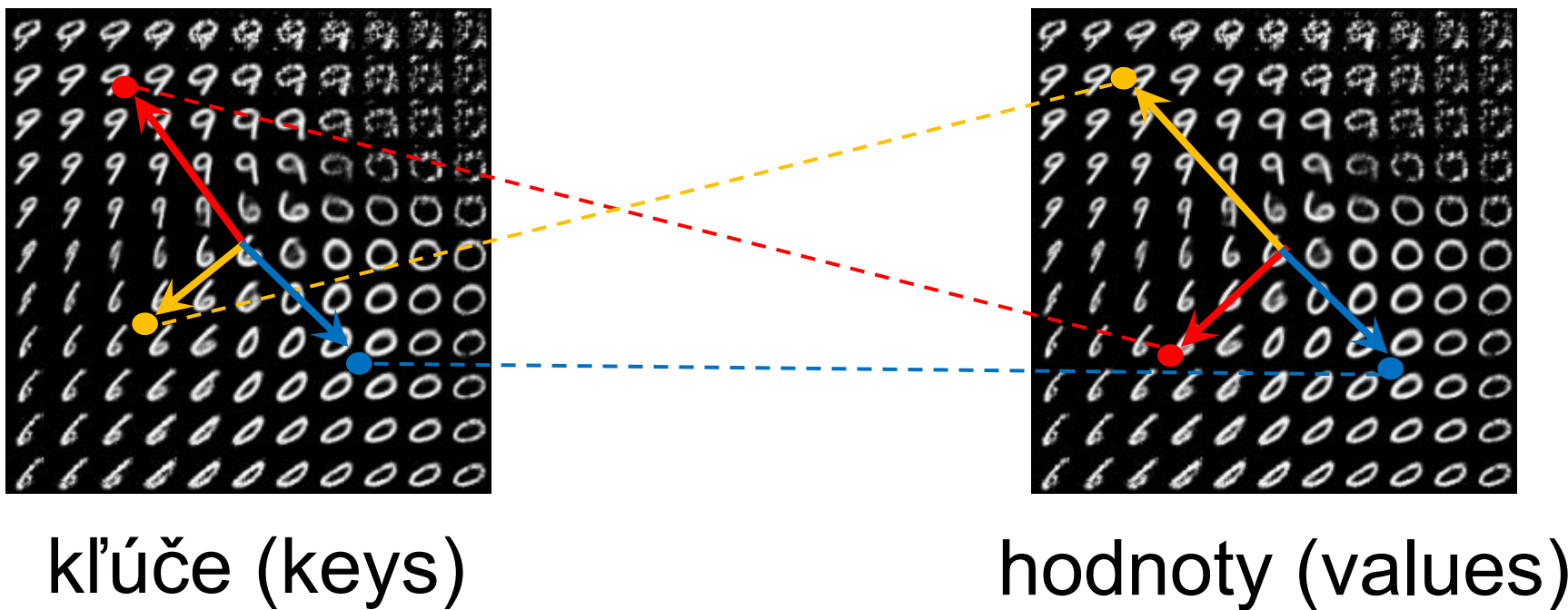
Mapovanie latentných priestorov

- Máme natrénovaný autokóder číslíc 0, 6, 9
- Ako ho prerobíme na model, ktorý číslice „obracia“ o 180° ?



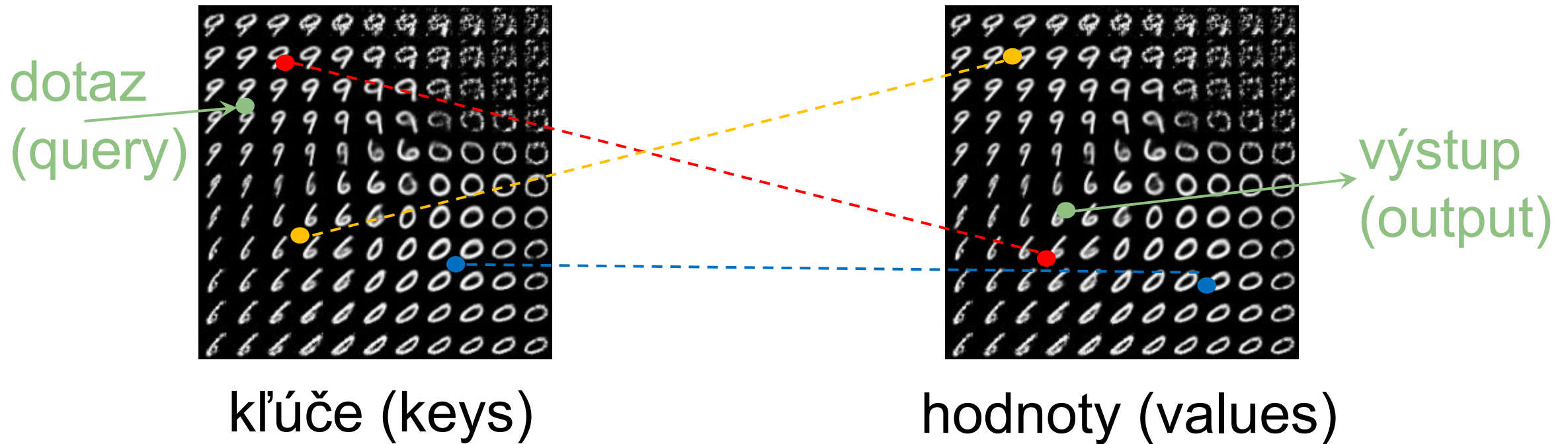
Mapovanie latentných priestorov

- Potrebujeme mechanizmus, ktorý z málo asociácií vyráta všetky



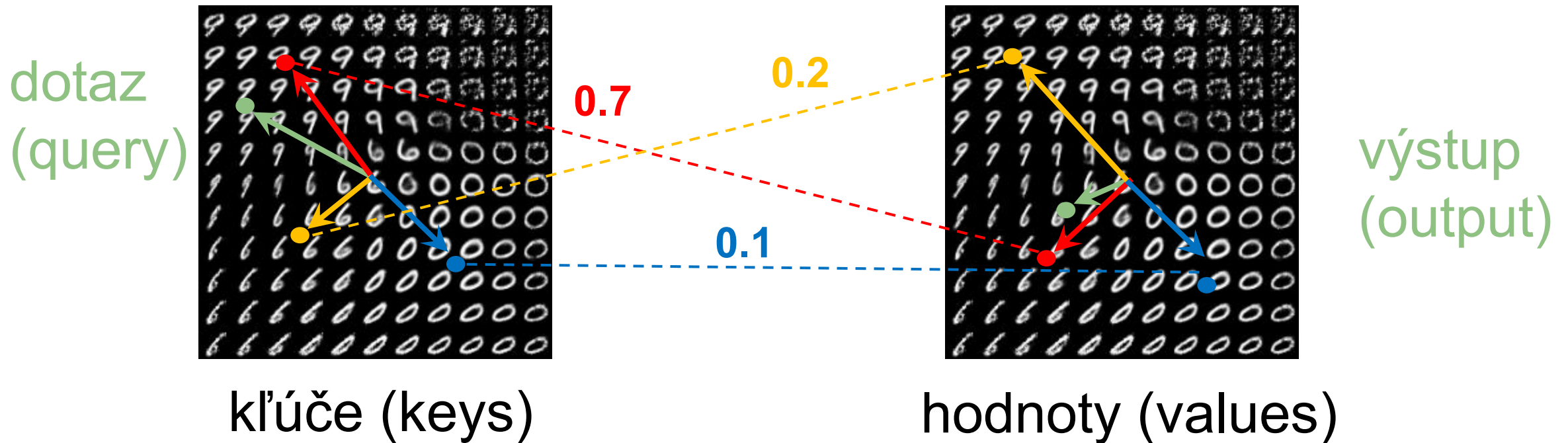
Mapovanie latentných priestorov

- Ako pomocou párov kľúč – hodnota, spočítať správny výstup (output) pre ľubovoľný dotaz (query) ?



Mapovanie latentných priestorov

- dotaz „namiešame“ z kľúčov a výstup urobíme ako analogickú „miešatinu“ hodnôt. Koeficienty „miešania“ dávajú v sume 1.0 a nazývame ich pozornosťné váhy (attention weights)



Mapovanie latentných priestorov

- Tieto váhy sa ľahko spočítajú, pokiaľ v latentnom priestore dimenzie d panuje kosínusová podobnosť.



$$K = \begin{pmatrix} k_0 \\ k_1 \\ \dots \\ k_{l-1} \end{pmatrix}$$

klúče (keys)

$$V = \begin{pmatrix} v_0 \\ v_1 \\ \dots \\ v_{l-1} \end{pmatrix}$$

hodnoty (values)

Mapovanie latentných priestorov



- Nakoľko sa query q podobá na kľúče $k_0, k_1, \dots, k_{l-1}$, sa dá vyjadriť ich skalárnymi súčinnami:

$$(q \cdot k_0, q \cdot k_1, \dots, q \cdot k_{l-1}) = qK^T$$

- Čím väčší je skalárny súčin, tým sú podobnejšie

Mapovanie latentných priestorov



- Vieme z
$$(q \cdot k_0, q \cdot k_1, \dots, q \cdot k_{l-1}) = qK^T$$
spočítať poznostné váhy?
- Samozrejme:
$$c = \text{softmax}(qK^T)$$

Mapovanie latentných priestorov

$$c = \text{softmax} \left(\frac{qK^T}{s} \right)$$



- pridaním škály s dokážeme regulovať koľko namiešame z podobnejších a koľko z odlišnejších kľúčov
- pri dostatočne malej škále dostaneme jednotku a nuly
- prirodzená škála je odmocnina z dimenzie

$$s = \sqrt{d}$$

Mapovanie latentných priestorov

$$\begin{aligned} o &= cV = c_0v_0 + c_1v_1 + \cdots + c_{l-1}v_{l-1} = \\ &= \left(\sum_{i=0}^{l-1} c_i v_{i,0}, \sum_{i=0}^{l-1} c_i v_{i,1}, \dots, \sum_{i=0}^{l-1} c_i v_{i,d-1} \right) \end{aligned}$$



- Výstup potom namiešame analogicky z hodnôt

Mapovanie latentných priestorov

- Dohromady teda dostávame vzorec



$$o = \text{softmax} \left(\frac{qK^T}{\sqrt{d}} \right) V$$

Mapovanie latentných priestorov

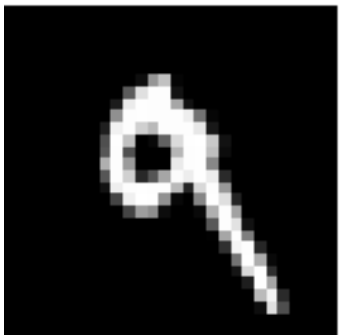


- A pokiaľ je dotazov viac naraz

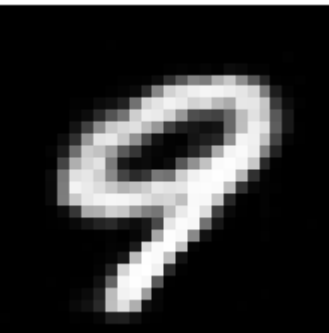
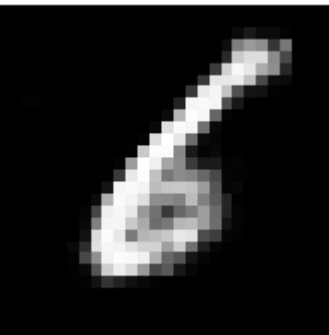
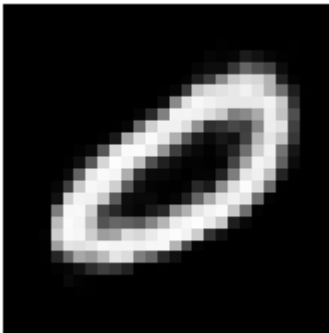
$$Q = \begin{pmatrix} q_0 \\ q_1 \\ \dots \\ q_{k-1} \end{pmatrix}$$

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

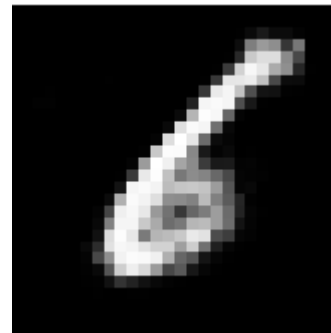
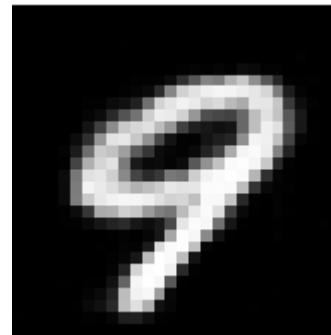
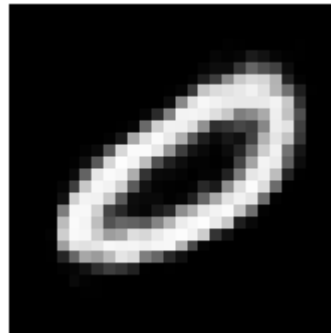
query



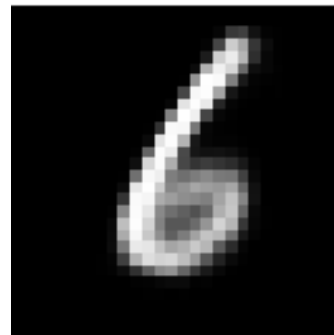
keys



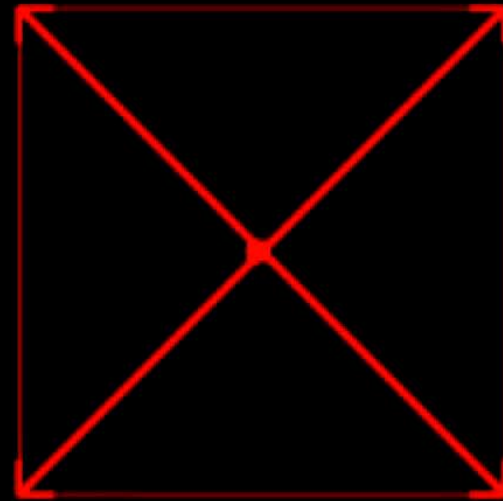
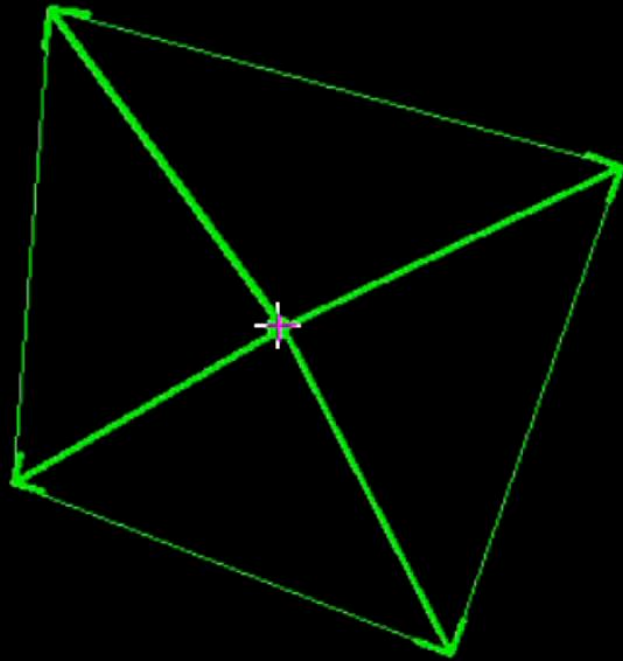
values



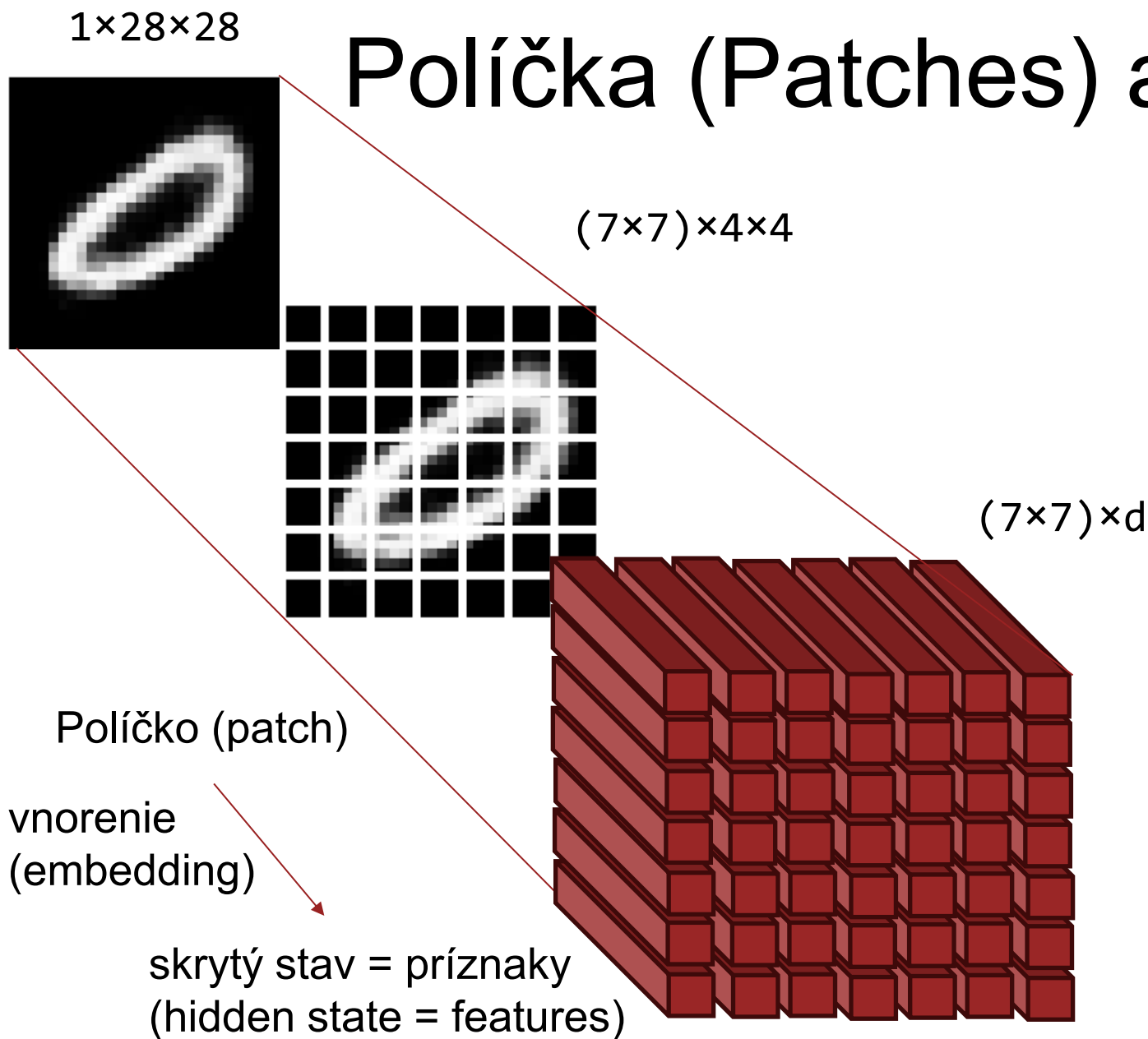
output



scalin...or: 10

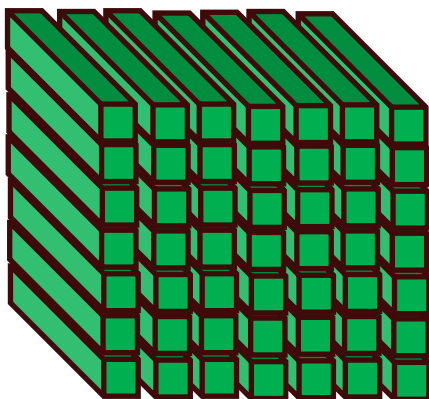
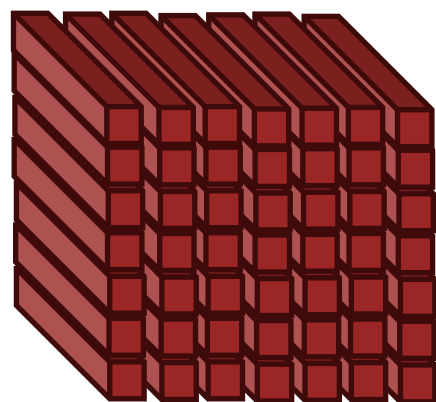


Políčka (Patches) a Príznakové mapy



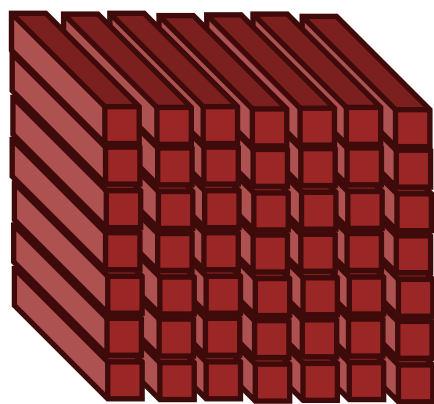
- Úspešnosť postupu vnorenie – vynorenie (embedding – wipeout) aplikovaného na obraz ako celok je limitovaná, avšak môžeme ju s úspechom použiť na jeho políčka (patch)

(Trénované) pozičné kódovanie

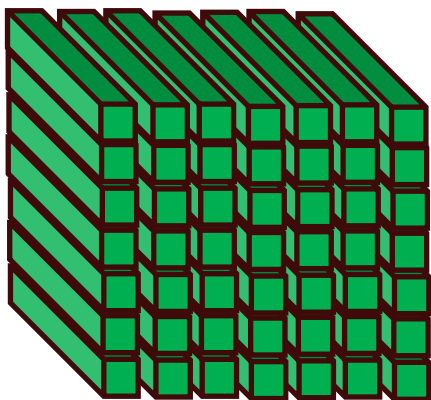


- Vnorenie políčka je rovnaké, bez ohľadu na jeho pozíciu na obraze
- Pozícia políčka však má na jeho význam určitý vplyv
- Preto sa zavádza pre každú pozíciu parameter o veľkosti vnorenia, ktorý sa k vnoreniu pripočítava
- Jeho hodnota sa trénuje z dáta spolu s ostatnými parametrami siete

jabĺčka + hrušky ?

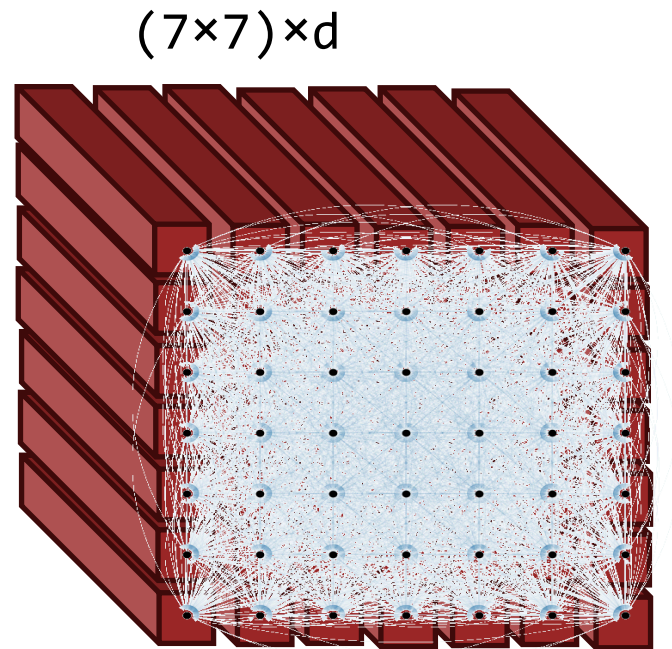


+



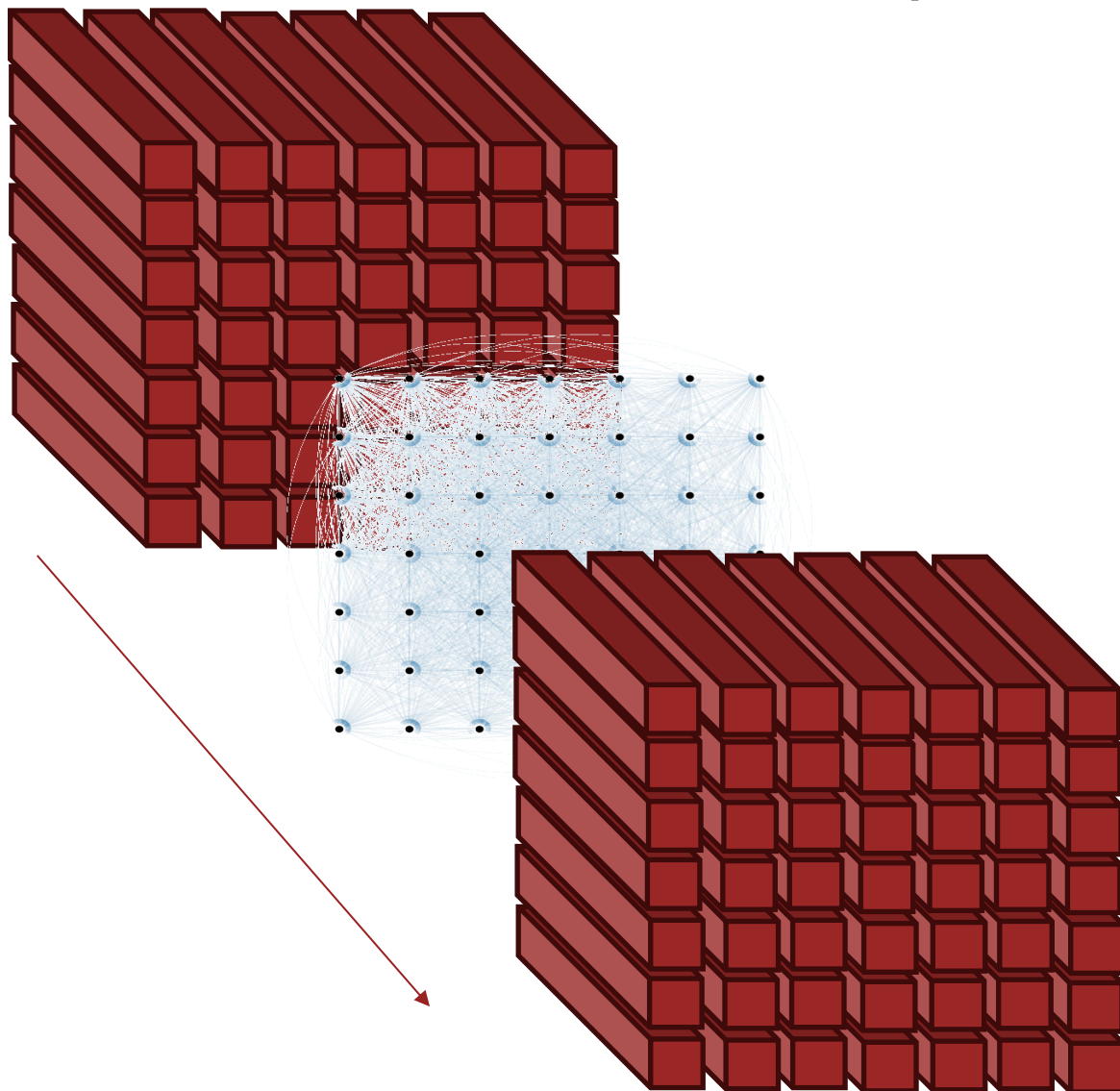
- môže toto fungovať? Môže.
- pri neurónovej sieti nemáme záruku, že sa ju podarí natrénovať
- Ale ak sa to podarí a dizajnovaná je tak aby dobre fungovala práve vtedy, keď ščítavame jabĺčka + jabĺčka, tak sa v priebehu tréovania z jabĺčok a hrušiek stanú jabĺčka a jabĺčka

Konkretizácia významu skrytých stavov podľa kontextu



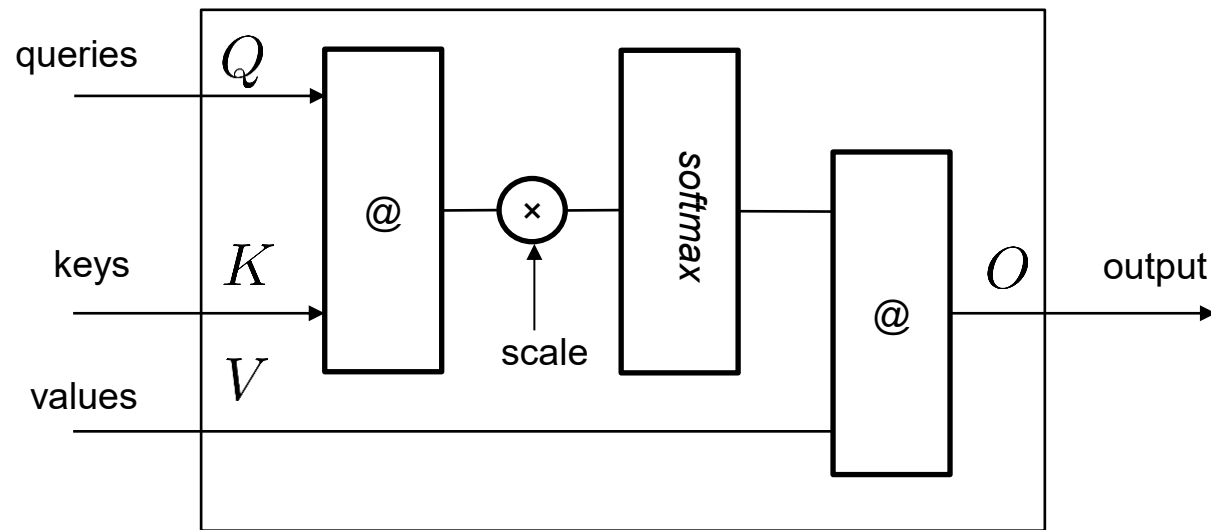
- Každé políčko je opísané d príznakmi
- Dokážeme príznakový vektor políčka upresniť podľa kontextu, ktorý predstavujú ostatné príznakové vektory?
- ak zariadime, aby podobné embeddingy mali tie políčka, ktoré sa v datase vyskytujú na tých istých obrázkoch, tak jeden prostriedok už poznáme

Autopozornosť (Self-Attention)



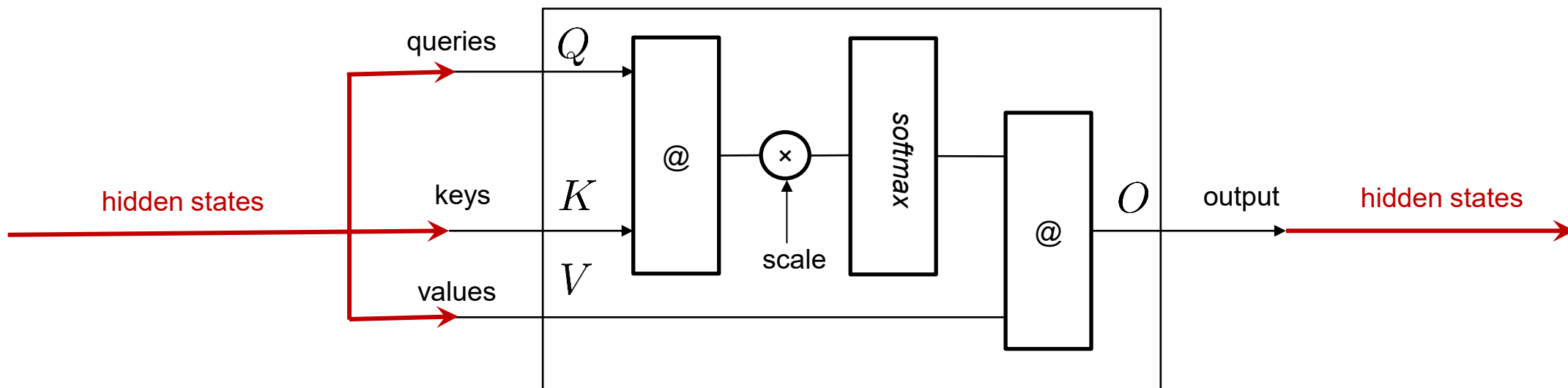
- Poslúži na to pozornosťný mechanizmus každého s každým
- Každý skrytý stav trochu namiešame s tými skrytými stavmi, ktoré sa s ním vyskytujú v tom istom kontexte (a teda sú podobné v zmysle kosínusovej podobnosti)

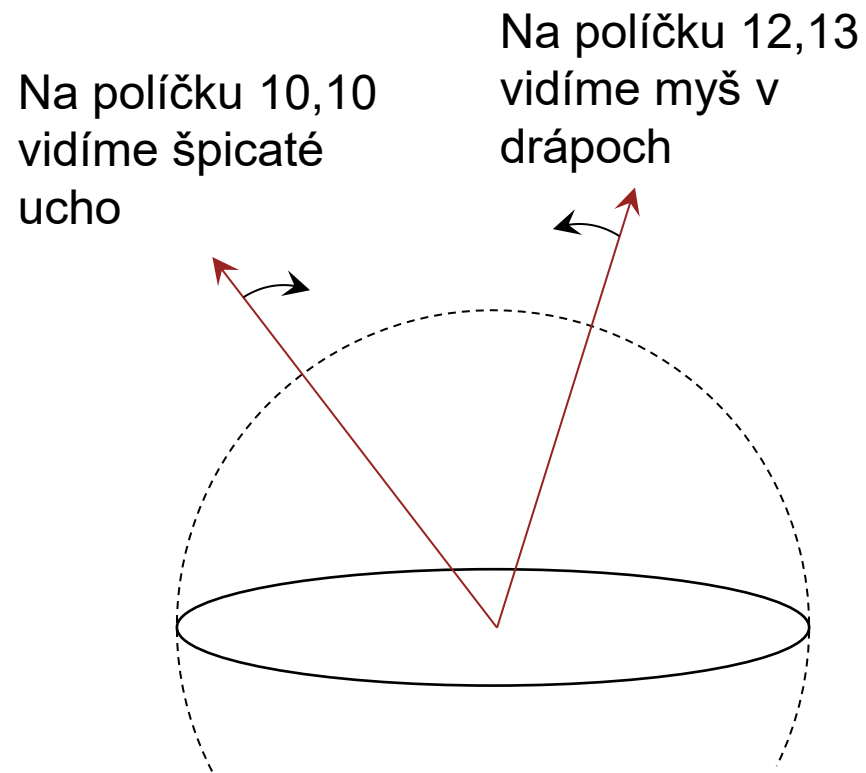
Pozornost' (Attention)



$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

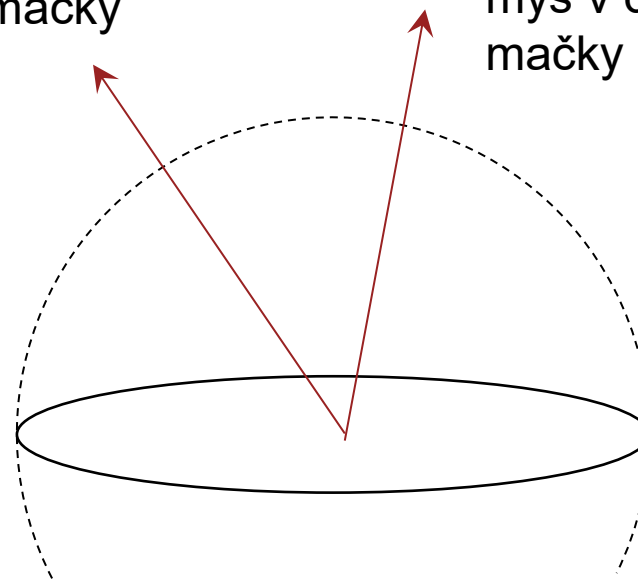
Autopozornost' (Self-Attention)





Na políčku 10,10
vidíme špicaté
ucho mačky

Na políčku 12,13
vidíme ulovenú
myš v drápoch
mačky



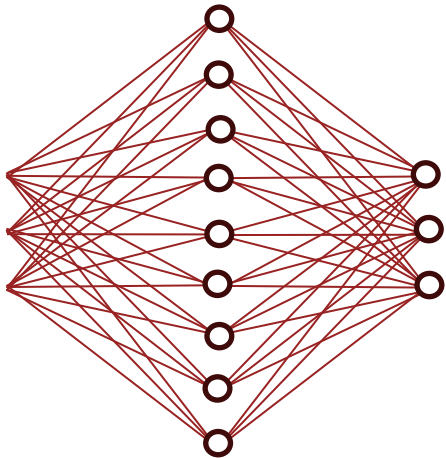
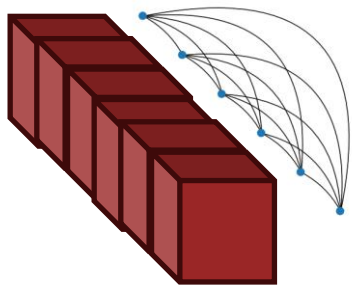
- *d* rozmerný priestor reprezentuje významy políčk
- je utriedený tak, že čím menší uhol reprezentácie zvierajú, tým častejšie sa vyskytujú v tom istom kontexte a tým viac sa významovo ovplyvňujú
- počítame ako sa má každý skrytý stav v priestore významov posunúť podľa kontextu, t.j. prítomnosti ostatných skrytých stavov
- tým skryté stavy získavajú globálnejší význam

Vytiahnutie sa za vlastné vlasy



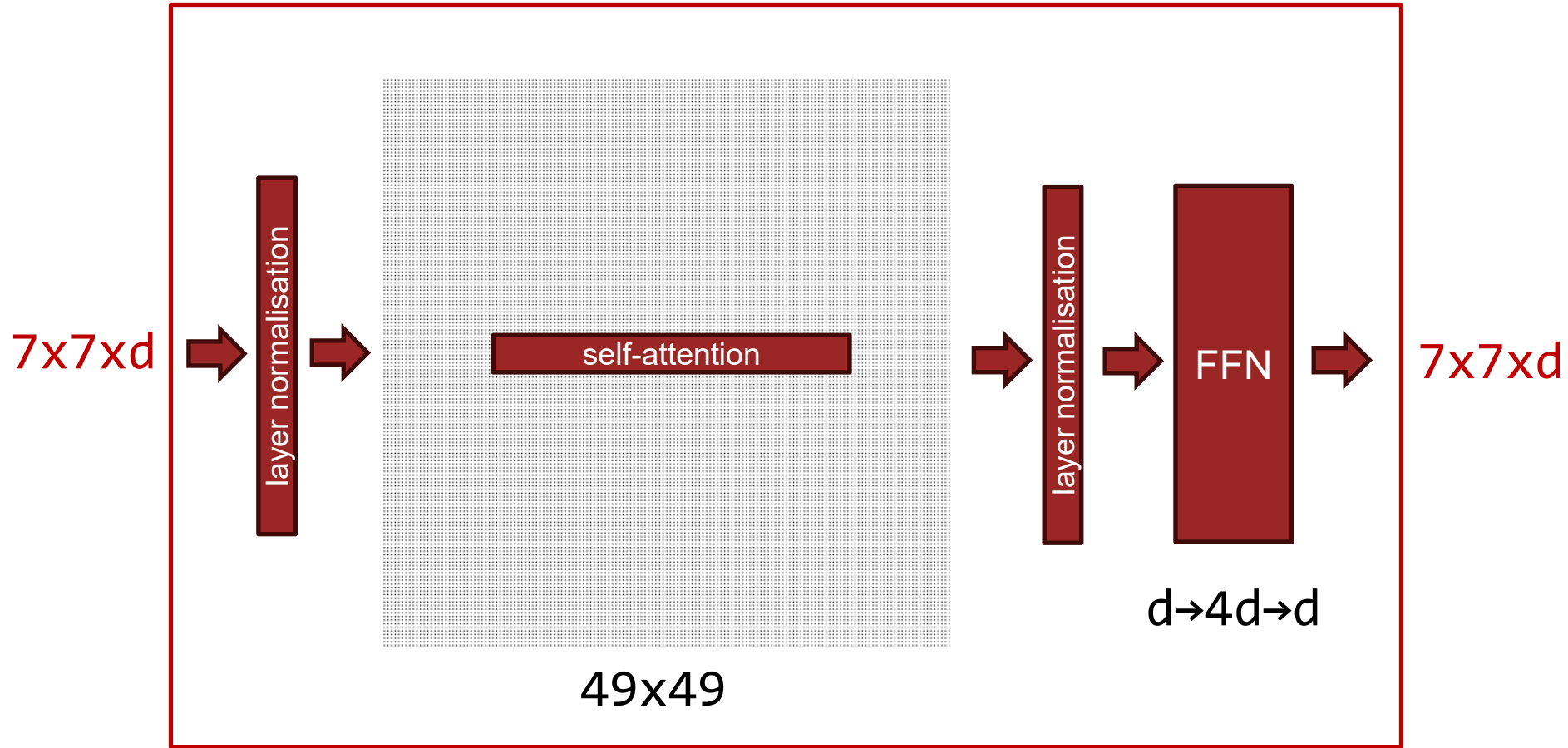
- Čo zaručí, že priestor významov skrytých stavov bude správne usporiadaný?
- Zariadime to tým, že transformer nadizajnujeme tak, aby fungoval práve vtedy
- Ak sa nám ho poradí natrénovať, tak priestor bude usporiadaný
- Tento trik sa v dizajne neurónových sietí hojne používa a FUNGUJE 😲

Feed Forward network (FFN)



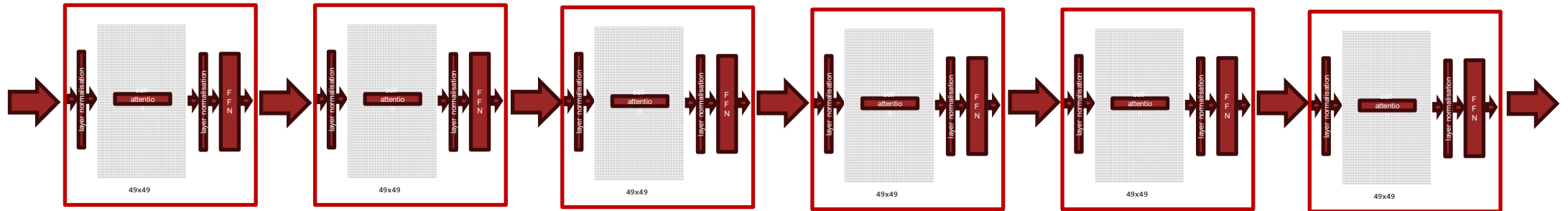
- Zmysel má tiež nechať ovplyvňovať sa príznaky navzájom
- Na to použijeme perceptrón, ktorý ma toľko vstupov i výstupov koľko je príznakov
- Každý skrytý stav tento perceptrón spracúva nezávisle
- Pritom sa do spracovania vnáša nelinearita, obľúbené aktivácie v skrytej vrstve tohto perceptrónu sú GELU a SiLU

Transformačný blok



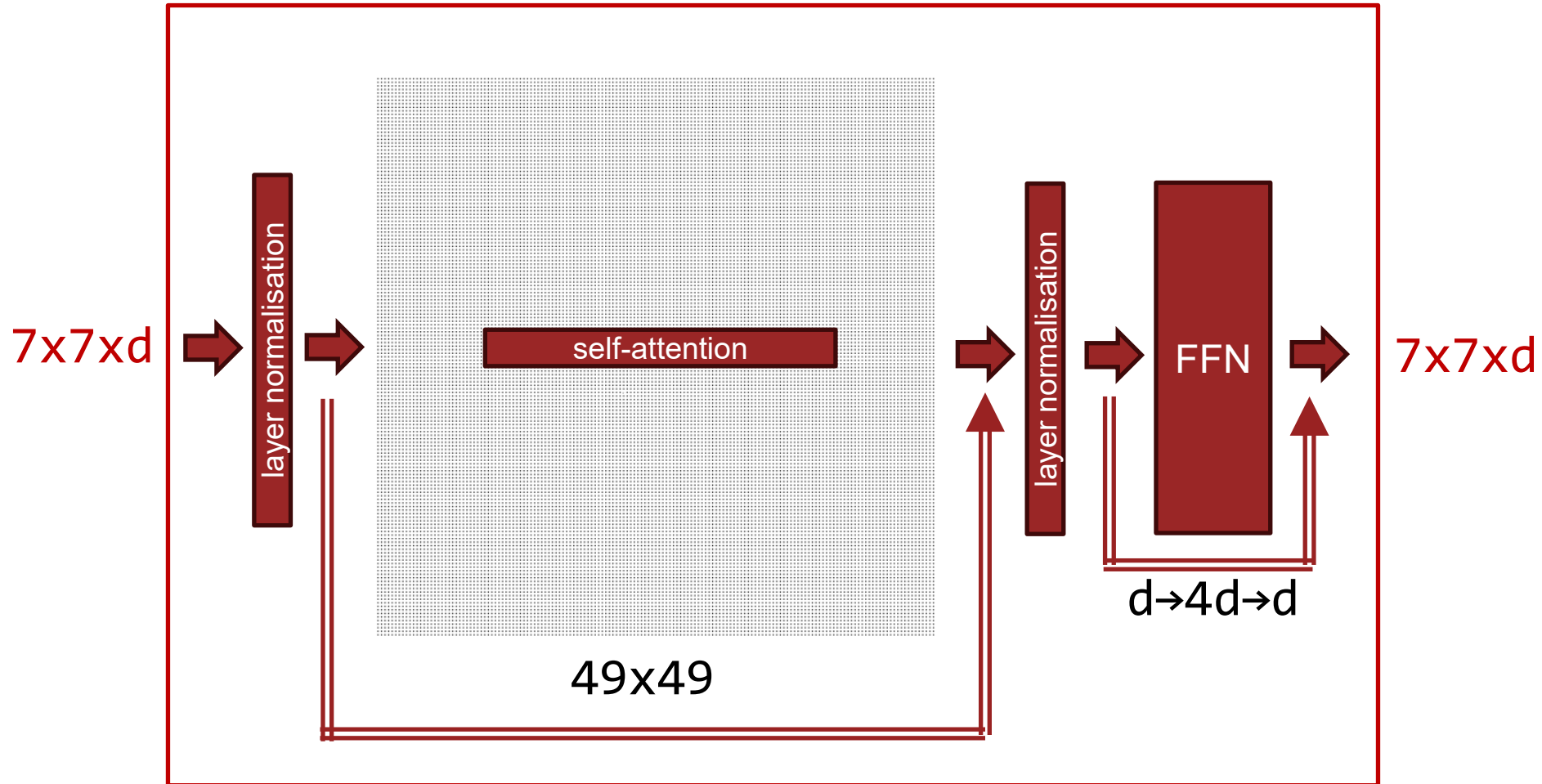
Potreba reziduálov

- Ponúka sa možnosť napojiť za seba niekoľko transformačných blokov a tak získať precíznejšiu reprezentáciu políček

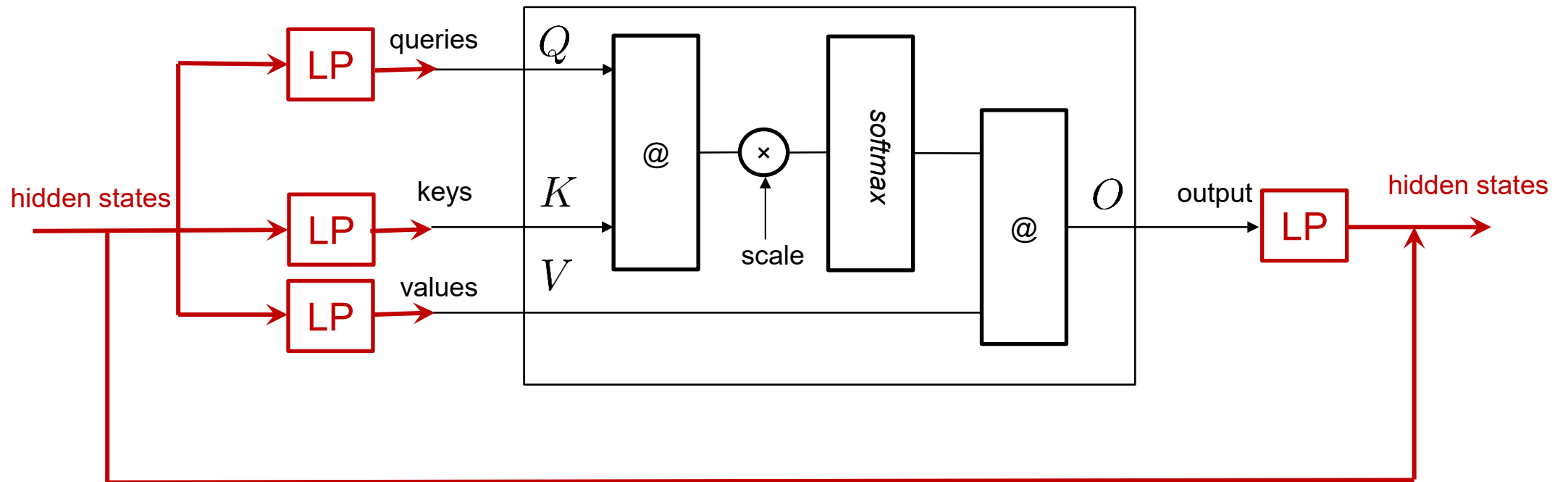


- Ale beda! Takto hlbokú sieť sa nám už nepodarí natrénovať
- Vieme však už čo s tým: potrebujeme do siete vložiť reziduálne spojenia

Transformačný blok

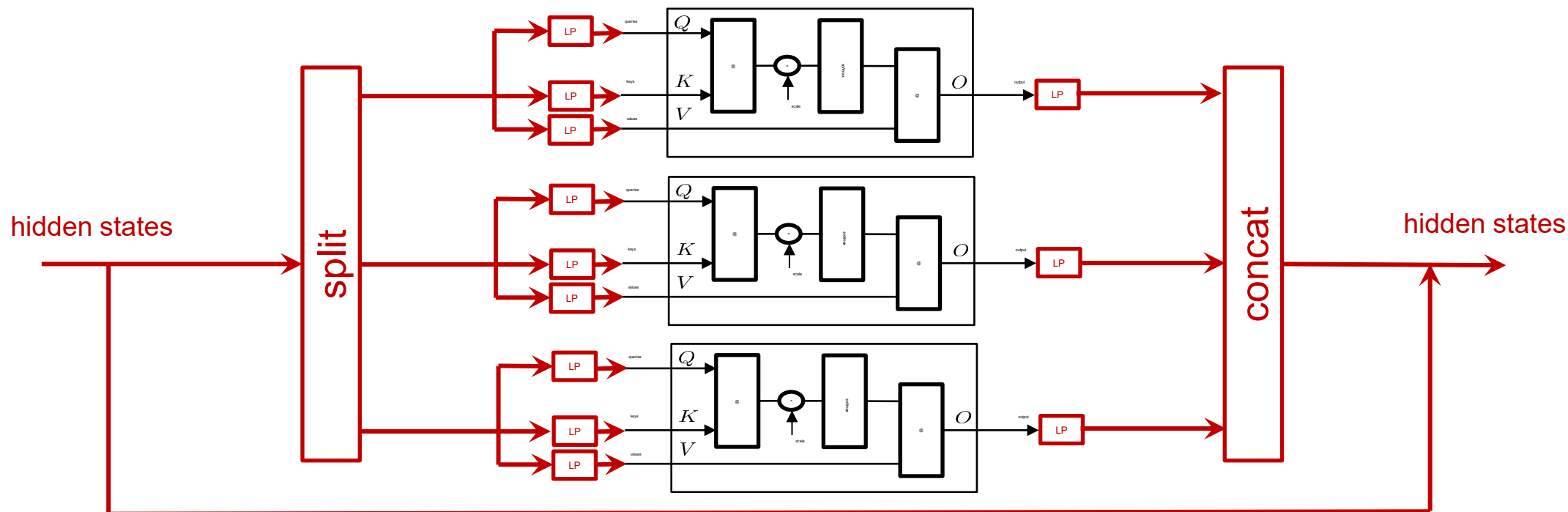


Reziduálna autopoornosť (Residual Self-Attention)



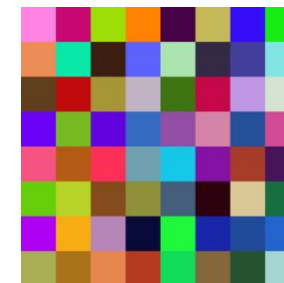
- Nepočíta zmenený skrytý stav, ale len jeho zmenu

Reziduálna viachlavá autopozornosť (Residual Multi-head Self-Attention)

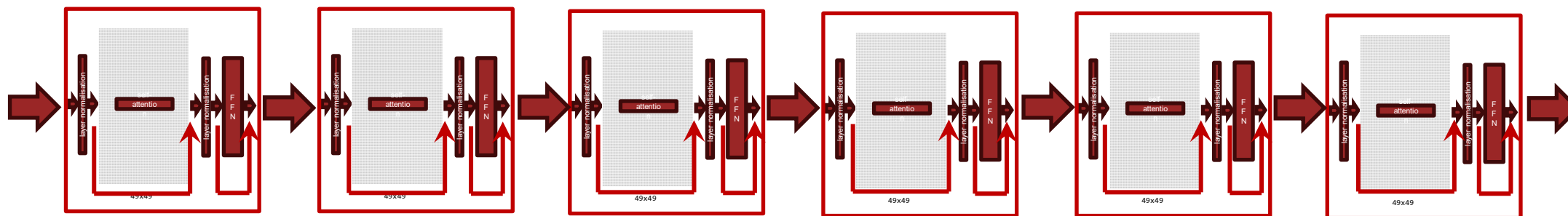


- delí príznaky na skupiny, ktoré sa navzájom neovplyvňujú

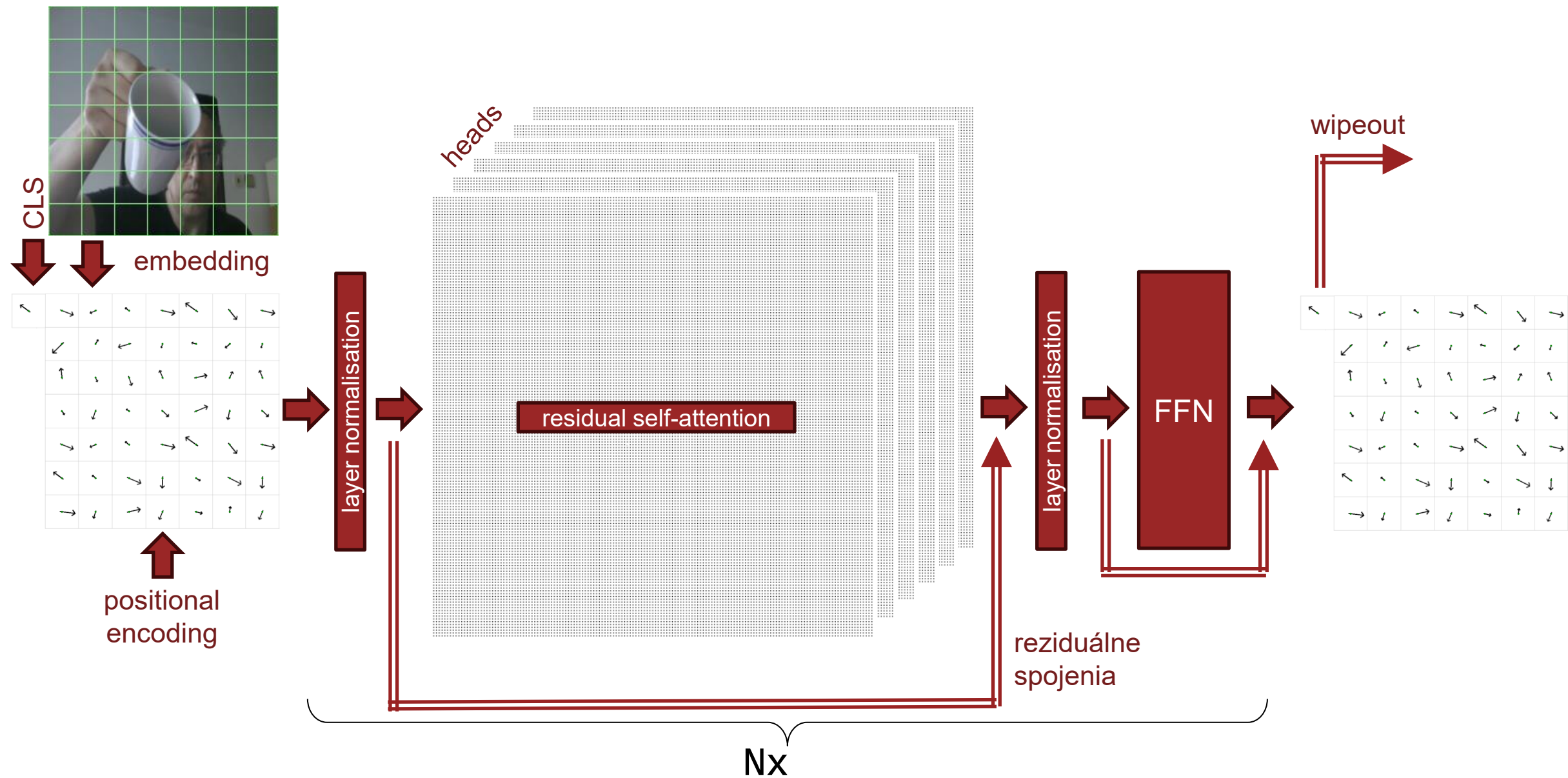
Potreba CLS (klasifikačný stav)



- Zatiaľ máme skryté stavy napríklad $7 \times 7 \times d$
- Z čoho urobíme vynorenie (wipeout) ?



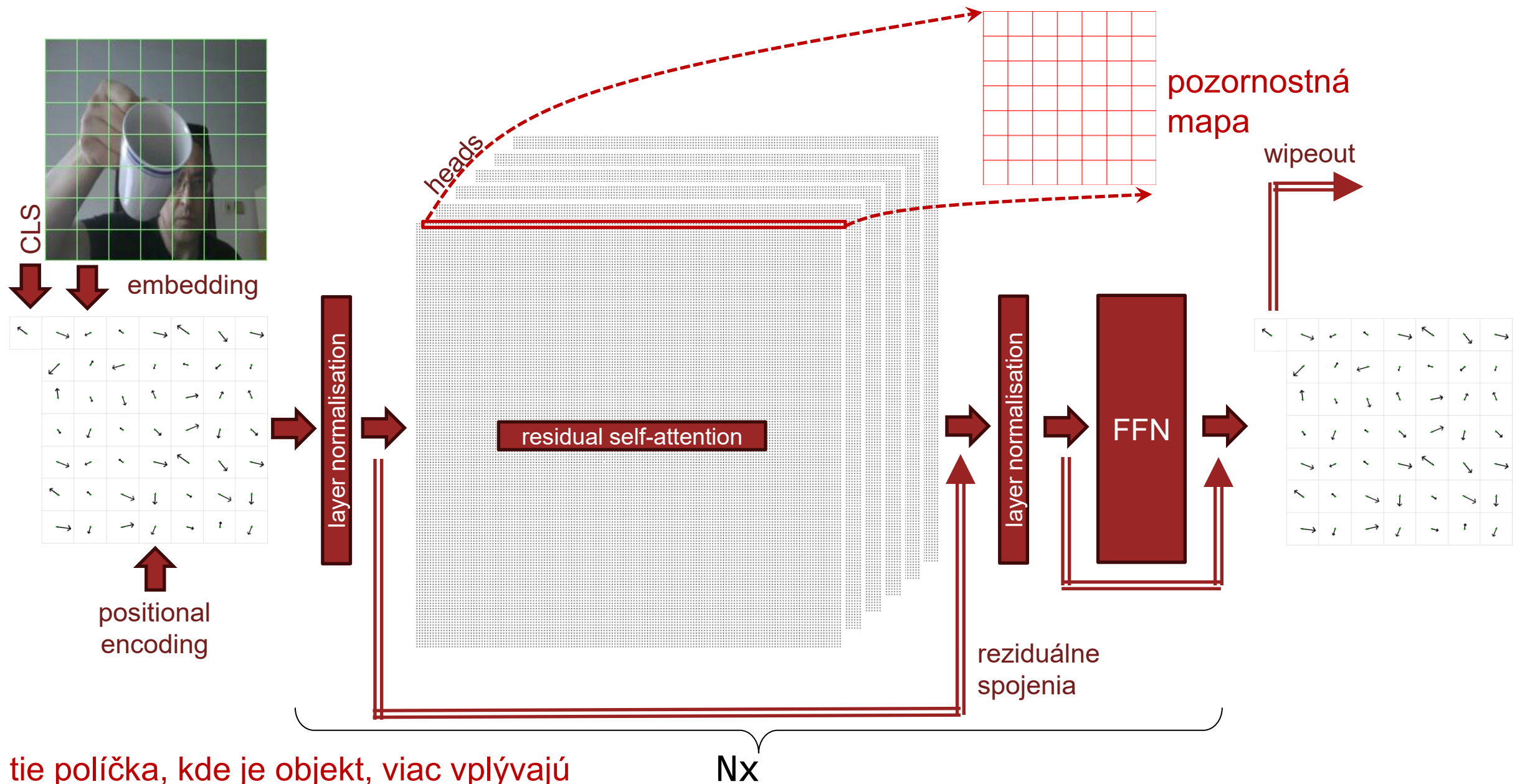
- Z jedného z nich? Zo všetkých?
- Najčistejšie bude, keď jeden skrytý stav pridáme a poveríme ho vychytaním globálneho významu
- Jeho iniciálnu hodnotu necháme sa systému naučiť, z koncovej urobíme wipeout



Vizuálny Transformer

Čím je ViT výnimočný?

- ViT je klasifikátor, ale nielen to
- ViT je zároveň detektor, ktorý poskytuje masku videných objektov
- Všetky predchádzajúce detektory objektov sa opierali o hrubú silu v tom zmysle, že najprv vytvorili klasifikátor (t.j. stroj, ktorý povie čo vidí) a potom ho pustili na veľa rôznych miest na obraze.
- **ViT to robí inak. Z toho, že vie čo vidí, vie aj kde to vidí**



tie políčka, kde je objekt, viac vplývajú
na určenie o aký objekt ide, vďaka
čomu vieme aj, kde sa objekt nachádza

Pozornostné mapy

pozornostné mapy

12 hláv

12 vrstiev

Prečo sú
najlepšie
mapy v
strede?

